Predicting production runtime from a development machine

This is the design and the current status of the four moves that replace the project's original cross-environment predictor. It exists because the original one **did not do the thing the project set out to do.**

Why the original approach could not work

The thesis is "predict production runtime from the developer's machine." The one experiment that tested exactly that — Apple M1 (arm64) as development, five EC2 instances as production — produced this (`results/tier1_laptop_dev.md`):

laptop(arm64) → EC2(x86_64)	**MAPE**
`cmp_only`, compute probed at `nproc`	324.24%
`cmp_only`, probed at the achievable DOP	153.2%
assume production = development (do nothing)	**123.36%**
for reference, EC2 → EC2 within the training hull	*20.06%*

The model was worse than doing nothing. Three root causes, none of which more machines would fix:

1. **The probe measured the wrong quantity.** The compute axis was FP-vector GFLOPS. TPC-H is hash-table probes, sorts, aggregation and sequential scans — integer and memory work. Nothing in the workload issues the instructions the probe measured.
2. **The signature was not two-dimensional.** Its two axes correlate at r = +0.958 with 97.9% of variance on one principal component, and it tracks core count at r = +0.841. It was a "how big is this machine" proxy that happened to correlate on x86.
3. **It could not describe managed Postgres at all.** The probe is a C program needing a shell and a compiler. RDS, Aurora and Cloud SQL give you neither.

The four moves

1. Measure the operators, in SQL — `scripts/collection/operator_probe.py`

Six axes, measured by running the engine against its own primitives:

axis	what it measures
`seqscan`	tuples/s through the scan + filter path
`hashjoin`	tuples/s to build and probe a hash table
`sort`	tuples/s through the sort path
`hashagg`	tuples/s through hash aggregation
`nestloop`	tuples/s through indexed nested-loop lookup (random access, TLB)
`parallel_seqscan`	the same scan with parallelism on — yields `parallel_speedup`

Because it is only SQL it **works on managed Postgres**, and because it is the same engine running the same code path it **crosses an ISA boundary** rather than relying on a proxy that happens to correlate.

The axes decouple, which is the point. Dropping `work_mem` from 64MB to 1MB on one machine left `seqscan` at **1.049×** and cut `hashjoin` to **0.580×** — a 1.81× spread that a single capability scalar cannot represent.

Two guards, both of which caught real errors:

- **Operator verification.** Each probe checks that the planner actually chose the operator it names. This caught the `nestloop` probe silently measuring a hash semi-join and reporting 5.2M tuples/s instead of the true 1.25M.
- **Repeatability gate (`MAX_CV = 0.15`).** A capability probe is only a capability measurement on an idle machine. Probing this host while a container sweep ran on it moved `seqscan`'s cv from 0.008 to 0.337 and the derived parallel speedup from **2.47× to 1.14×**. Records that breach the gate are flagged `reliable: false` and `cetp.cross_env.load_rates` refuses them.

2. Scale per operator — `cetp/operator_model.py`

```
scaling = Σ over (axis, parallel?)   w × rate_dev(axis)·s_dev / (rate_prod(axis)·s_prod) ×
card
T_prod  = T_dev × scaling
```

`w` is development's **wall-time** share for that bucket, so the weights sum to 1.

Why a weighted ratio and not a sum of scaled node times. Under a `Gather`, PostgreSQL reports each node's `Actual Total Time` averaged across participants, so node self-times sum to total **CPU** time — measured at 2.85× wall on a parallel TPC-H q19. Summing scaled self-times would predict ~3× too high. Normalising to weights makes that factor cancel.

Two corrections that are easy to get wrong, and both were:

- `Actual Total Time` is **cumulative** (subtract children) and **per-loop** (multiply by `Actual Loops`). An inner index scan reporting 0.31 ms over 485 loops costs 150 ms.
- The parallelism correction keys on **"below a `Gather`"**, *not* on `Parallel Aware`. `Parallel Aware` marks only nodes that *partition* work. On q19 the `Nested Loop`, inner `Aggregate` and `Index Scan` all report `loops=3` while only the `Seq Scan` is `Parallel Aware`. Keying on the flag corrected 46% of the plan's time instead of 100%.

The effect of getting the parallelism axis right, on the one pair measured so far (8-core host → 1-CPU container, TPC-H q19, true ratio 3.011×):

variant	predicted scaling	error
serial axes only	1.080	64.1%
+ parallelism, keyed on `Parallel Aware`	1.493	50.4%
+ parallelism, keyed on `under_gather`	**2.664**	**11.5%**

3. Take the plan from production — `scripts/collection/predict_prod.py`

`EXPLAIN` **without** `ANALYZE` on production plans the query and executes nothing. That plan is then forced on development with `pg_hint_plan` and run with `ANALYZE` there.

This eliminates plan divergence **by construction** rather than modelling it. The causal decomposition (`results/tier4_plan_drift_causal.md`) measured plan divergence at **70.5%** of the cross-environment difference (1.673× against 1.145× for settings) whenever the plan changed. Production's row estimates also give the data-volume ratio without touching production data.

If forcing fails — `pg_hint_plan` silently ignores hints it cannot express — the result carries `plan_forced=False`, trust drops to `LOW`, and the plan-divergence warning is printed. It is never hidden.

4. Few-shot, not zero-shot

From `results/tier3_cold_start.md`: 15.84% uncalibrated → **13.20% after one well-chosen query** → 12.57% at k=3, while k=12 chosen **at random** scored 13.45% and k=1 at random scored **19.51% — worse than not calibrating at all**. Selection matters more than budget, so `select_calibration_queries` spreads over predicted scaling and `fit_calibration` takes a **median**, not a mean.

What this still does not do

- **Concurrency.** Every number is single-client. Measured elasticity ranges +0.286 to +0.984 across queries, up to **8.57× at 8 clients**. Production load is an input the caller must supply; it is not inferable from a development machine. This is the single largest unmodelled term and it is larger than everything above.
- **The cardinality term is implemented but unvalidated.** Every container-grid cell holds identical SF1 data, so it is only ever exercised at ratio 1.0.
- **Cross-ISA is still unmeasured.** Containers on an arm64 host are arm64. Validating the ISA claim needs free x86 (GitHub Actions runners) and is not done.

Status — measured

Two evaluations. Full numbers in **results/operator_model.md** (container→container, 1232 pairs) and **results/laptop_to_prod.md** (a real laptop → 6 production machines, 132 pairs).

The thesis, achieved

laptop → production, zero-shot	MAPE
naive (assume prod = dev)	38.22%
operator model	**28.54%**

Measuring 5–8 queries on production takes it to **~23%**. The gain concentrates where capability genuinely differs: on the 1-CPU cells, 62.58% → 25.18% and 59.78% → 28.84%.

What each move is actually worth

move	validated?
1. operator probe (SQL, works on RDS)	**yes** — signature is now 4-dimensional (PC1 59.4%, mean \
1b. **parallelism axis**	**yes — this is the result.** +48.80 pp over the identical model without it, CI [+38.60, +60.13]
2a. **localising parallelism to the plan's parallel part**	**yes** — +6.91 pp over applying it whole-query, CI [+0.95, +14.73]
2b. per-operator *rate* weighting	**no** — a single serial scalar with localised parallelism scores 22.23% against the full model's 22.30%; Δ −0.08 pp, CI [−0.73, +0.44]
3. plan from production	**not testable here** — 0 of 22 queries changed plan across cells
4. few-shot calibration	**modest** — 28.54% → ~23% at k=5–8, but **k=1 is worse than not calibrating** (33.74%)
missing: I/O axis	**diagnosed as the residual error** — the probe measures no storage throughput

Cross-ISA: measured, and it fails without calibration

results/cross_isa_prediction.md — 176 pairs, 8 production machines, one of them a real x86_64 AMD EPYC:

population	naive	operator, zero-shot	operator, few-shot
same ISA (7 arm64 cells, 154 pairs)	37.39%	**29.31%**	~25%
cross ISA (**4** x86_64 machines, 88 pairs)	21.7%	**68.9%**	**~22%**

Replicated 4 out of 4 on independent GitHub Actions hosts (66.15 / 67.26 / 69.44 / 72.84%), with signatures agreeing to within 5% of each other — so this is a systematic property of the architecture boundary, not probe noise.

The probe transfers within an architecture and not across one. It reports the M1 faster on every axis and predicts x86 is 1.667× slower; it is actually 1.044×. Not operator-mix divergence (that was tested and refuted — 1.365× cross-ISA against 1.371× within the arm64 grid) but the **absolute scale**: tuples/s on a synthetic operator is not comparable across microarchitectures. Same defect as the original FP-GFLOPS signature; five architecture- sensitive scalars instead of one did not fix it.

Calibration is the fix, and it is mandatory across an ISA. Five production queries take the x86 cell from 72.84% to 23.38%, near its 16.75% floor — but only to *parity* with naive's 21.41%, not past it.

The honest headline

Parallelism is the finding; per-operator detail is not. A capability signature without a parallelism term is level with or worse than doing nothing (71–74% against naive's 72.52%), which reproduces the original laptop→EC2 failure (324% against 123%) and explains it: a CPU quota does not slow a single-threaded operator, so the serial axes span only 1.02–1.13× across the grid while parallel speedup spans 3.66×.

Adding the parallelism axis and **localising it to the nodes below a** `Gather` gets the full result. Weighting per operator on top of that adds nothing measurable. **That is a simplification result:** one serial capability scalar + one parallel speedup + the plan's parallel/serial wall-time split is sufficient on this evidence. The five-axis probe is how the speedup is measured and may matter across ISAs, but its per-axis detail is not carrying the prediction today.

A correction worth recording

An earlier pass of the laptop→production evaluation, run on probes taken while the machine was busy, reported zero-shot at **47.36% against naive's 41.11%** — the opposite conclusion, that the model was worse than doing nothing. Re-probing every cell on an idle machine moved three cells from failing the repeatability gate to passing it and reversed the verdict. **Probe hygiene is load-bearing, not a detail.** The `MAX_CV = 0.15` gate and `cetp.cross_env.load_rates` refusing flagged records exist because of this.

What to do next, in order

1. **Add an I/O axis to the probe.** The diagnosed residual: the probe is cache-resident (~120 MB tables against a 128 MB buffer pool) while TPC-H reads ~850 MB past it, so over-prediction correlates with each query's I/O fraction (Spearman r = +0.302). Needs a probe table several times `shared_buffers`.
2. ~~**Get a second ISA.**~~ **DONE, and the answer is NEGATIVE** — see results/cross_isa_operator_rates.md. An x86_64 AMD EPYC 9V74 runner was probed via `.github/workflows/x86-operator-probe.yml`. Crossing arm64 → x86_64 perturbs the operator mix by **1.365×**, while the worst pair *within* the arm64 container grid perturbs it by **1.371×** — no more than changing a CPU quota on one machine. **The stated motivation for per-operator rates does not survive measurement**, so the 59.7% cross-ISA floor is not caused by operator-mix divergence. With the grid ablation (−0.08 pp, CI [−0.73, +0.44]), two independent experiments now agree that per-operator *rates* are not the useful part of this model; the parallelism term and its plan-structure localisation are. **What explains the floor is now the open question.**
3. **Vary settings, not just quotas,** to create plan divergence and make move 3 testable.
4. **Validate the cardinality term** with a second scale factor.
5. **Make concurrency an explicit input.** Still the largest unmodelled term (8.57× at 8 clients) and it dwarfs everything above.